

Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Sınav bilgilendirmesi
 - Sınav haftasının bir sonraki hafta olduğu, dersin sınavının programa göre cuma günü yapılacağı.
 - Sınav formatı: klasik sınavdan öte, şıklı sorular veya boşluk doldurma tarzında olacağı; belki küçük bir kod parçasında boşluk doldurma olabileceği.
 - Belirli bir terminolojide kullanılan ifadenin bilinmesi ve açıklamasının karşılığında ne olduğuna dair yazılması isteneceği.
 - Bilgi ezberi beklenmediği, sürenin çok uzun olmayacağı belirtilmiştir.
 - Sınava hazırlanmak için developer.android.com üzerinden ilgili chapter'ların okunması önerilmiş, ek farklı bilgi veren siteler de kullanılabileceği belirtilmiştir.
 - Sınavda Dependency, Gradle, RecyclerView vs ListView farkı, aktivite tipleri, aktiviteler arası gezinme komponenti, bilgi taşıma gibi temel konuların sorulacağı.
- Liste yapıları: ListView, GridView, RecyclerView, ScrollView
 - ListView: en çok kullanılan yapı ancak son 4-5 senede yerini RecyclerView'a bırakmıştır.
 - RecyclerView'ın daha fazla özellik sunması ve performans artırma noktasında ciddi katkılar sağlaması.
- ListView vs RecyclerView temel farkları
 - ListView'da tüm item'lar aynı anda telefon üzerine yüklenir; bu yükleme sırasında belli bir bekleme süresine neden olur.
 - RecyclerView'da ise aşağı kaydırıldıkça item'lar yüklenir; bin item'dan on tanesini gösteriyorsanız bir sonraki on tanenin henüz çekilmemiş olması anlamına gelir.
 - RecyclerView, recycling (geri dönüşüm) mantığını kullanarak eski item view'u çöpe atmayıp yeni gelen item için kaynak olarak kullanır; büyük performans ve kaynak farkı yaratır.
 - Hem verinin görsel olarak sunulmasında hem de RAM ve pil tüketiminde avantaj sağlar.
- ViewHolder pattern
 - ListView ve RecyclerView'in her ikisinin de kullanabildiği bir pattern.
 - RecyclerView'da ViewHolder pattern'i zorunludur; ListView'da opsiyoneldir.
 - ListView'da ViewHolder pattern'i kullanılırsa performans önemli ölçüde RecyclerView'a yaklaşır, ancak RecyclerView hâlâ daha iyi performans gösterir.
 - ListView'da standart yapıyla ViewHolder kullanılmazsa ciddi kullanıcı memnuniyetsizliği ve kaynak tüketimi sorunu ortaya çıkar.
- LayoutManager kavramı
 - ListView'da sadece dikeyde bir view oluşturma şansı vardır; başka bir opsiyonu yoktur.
 - RecyclerView'da üç farklı layout kullanma opsiyonu vardır: Linear Layout Manager (yatay ve dikey), Grid Layout Manager (galeri uygulamaları için ideal), Staggered Layout Manager (Pinterest benzeri, farklı boyutlarda item düzeni için).
- Item dekorasyonu ve animasyonu
 - ListView bu konuda oldukça zayıftır; opsiyonel olarak sunmamasının ötesinde customize etme imkanı da vermez.
 - RecyclerView'da item animator ve item decoration ile farklı item'lar tasarlanabilir ve anime edilebilir.

- Karmaşıklık: animation ve decoration işine girildiğinde belli oranda efor harcanması gerekir.
- OnItemClickListener
 - RecyclerView'da bulunan özellik; onClickListener'dan farklı yetenekler sunar.
 - Tek tip click işlemi yerine touch işleminde birden fazla touch tipini destekleyen listeler ile item üzerinde farklı aksiyonlar tanımlanabilir.
- setHasFixedSize metodu
 - RecyclerView'da performans iyileştirme metodu.
 - Eğer item'ların sayısı belli ve değişmezse (örneğin 200 ülke × 100 şehir = 20.000 item) kullanılır.
 - Bu sayede RecyclerView her seferinde aynı pozisyonu alacağı için daha iyi kaynak planlaması yapılır ve cache mekanizması oluşturulur.
- Ne zaman ListView, ne zaman RecyclerView?
 - Çok basit yapılar (Türkiye'nin 81 ili gibi) ve animasyon ihtiyacı olmayan durumlarda ListView + ViewHolder yeterlidir.
 - Daha karmaşık, animasyon/dekorasyon istenen, büyük veri setleri için RecyclerView tercih edilmelidir.
- RecyclerView ile liste yaratma adımları
 - XML içinde RecyclerView tanımlanır; dependencies'e com.android.support:recyclerview eklenmelidir.
 - Java tarafında: RecyclerView bileşeni, Adapter ve LayoutManager tanımlanır.
 - setContentView ile bağlama, findViewById ile ilişkilendirme yapılır.
 - setHasFixedSize, setLayoutManager, setAdapter çağrıları yapılır.
- Adapter kavramı
 - View (ön yüz) ile data source arasındaki veri akışını sağlayan parça.
 - RecyclerView Adapter'ı kullanılır; kendi tarafınızdan extend edilerek yazılır.
 - Controller ile karıştırılmamalıdır: Controller tüm işlemleri yönetir, Adapter sadece verinin view'a aktarımını sağlar.
- Adapter'ın temel metotları
 - onCreateViewHolder: her item için bir ViewHolder üretir; LayoutInflater.from.inflate ile yeni bir item görüntüsü oluşturulur.
 - onBindViewHolder: oluşturulan view'un içine data source'dan gelen veriyi position'a göre set eder.
 - getItemCount: data set uzunluğunu döndürür.
- MyViewHolder sınıfı
 - RecyclerView.ViewHolder'ı extend ederek oluşturulur.
 - Her item'ın görsel bileşenlerini (TextView, ImageView, Button vb.) barındırır.
 - findViewById ile XML'deki view'lar holder'a atanır.
- ViewHolder'ın dinamik oluşumu
 - "Creates only as many ViewHolders as are needed to display on screen portion of the dynamic content" ifadesiyle, RecyclerView sadece ekranda görünen kadar ViewHolder oluşturur.
 - Scroll edildikçe ekranın dışına çıkan item'ların ViewHolder'ları yeni gelen item'lar tarafından yeniden kullanılır.

Hocanın Özellikle Vurguladığı Kısımlar

- ViewHolder pattern'inin kritikliği
 - "ListView'un en büyük sıkıntısı ViewHolder'ı opsiyonel yapmasıdır" vurgusu; eğer kullanılmazsa ciddi performans düşüklüğü yaşanır.
 - Performans farkı sadece teorik değil, pratikte de gözlemlenebilir.
- Dependency'lerin eklenmesinin zorunluluğu
 - RecyclerView'ı kullanabilmek için dependencies bölümüne com.android.support:recyclerview-version-7 versiyonunun eklenmesi gerektiği; eklenmezse hata alınacağı.
- Core Android programlamanın derste tamamen öğretilmeyeceği
 - "Core Android programlamayı derste öğretmek gibi bir amacımız yok"; 3 saatlik haftalık dersle her sene değişen package yapısına sahip ortamda her şeyi öğretmek zor.
 - Amaç, temel bilgileri vermek ve ileride ilgi duyanların kullanmasını sağlamak.
- Adapter ve Controller farkı
 - Öğrencilerin adapter ile controller'ı karıştırması üzerine hoca, bunların farklı kav-

ramlar olduğunu açıkça belirtmiştir: Controller arka plan veri modeli ile ön yüz arasındaki tüm işlemleri yöneten kod, Adapter ise sadece RecyclerView/ListView'e özel veri aktarımını sağlayan yapı.

Kısa Tekrar Notları

- RecyclerView vs ListView: RecyclerView daha performanslı, ViewHolder zorunlu, layout yönetimi esnek (3 tip), animasyon/dekorasyon destekli.
- 3 LayoutManager: LinearLayoutManager (yatay/dikey), GridLayoutManager, StaggeredLayoutManager (Pinterest tipi).
- ViewHolder pattern: ListView'da opsiyonel, RecyclerView'da zorunlu.
- setHasFixedSize: item sayısı belli ise performans artışı sağlar.
- Adapter metotları: onCreateViewHolder, onBindViewHolder, getItemCount.
- Adapter: data source ile view arasındaki köprü; Controller ile karıştırılmamalı.
- Dependency: com.android.support:recyclerview.

Detaylı Açıklamalar

Dersin başlangıcında hoca, mikrofonların kapatılmasını hatırlatmış, sınav haftasının bir sonraki hafta olduğunu ve başarılar dilediğini belirtmiştir. Sınavın cuma günü yapılacağı, sınavın şıklı sorular ve boşluk doldurma tarzında olacağı, küçük bir kod parçasında boşluk doldurma olabileceği, belirli bir terminolojide kullanılan ifadenin bilinmesinin isteneceği söylenmiştir. Bilgi ezberi beklenmediği, çok uzun süren bir sınav olmayacağı belirtilmiştir. Sınavdan sonra ikinci ödevin verileceği, bu ödevin o sırada geliştirilen uygulamayla ilgili olacağı ve muhtemelen bir hafta-10 gün içinde teslim edileceği açıklanmıştır. Sınava hazırlanmak için developer.android.com üzerinden ilgili chapter'ların okunması, dependencies, gradle, RecyclerView vs ListView farkı, aktivite tipleri, aktiviteler arası gezinme komponenti, bilgi taşıma gibi konulara çalışılması önerilmiştir. Hoca, Core Android programlamanın derste tamamen öğretilemeyeceğini, 3 saatlik haftalık dersle her şeyin öğrenilemeyeceğini, amaçlarının kritik bilgileri vermek ve ileride ilgi duyanların kullanmasını sağlamak olduğunu vurgulamıştır. Slide'ların cuma gününe kadar paylaşılacağı, bunların developer.android.com'dan toplanmış seçmece slide'lar olduğu söylenmiştir.

Dersin ana konusuna, yani liste yapılarına geçildiğinde ilk olarak ListView, GridView, RecyclerView ve ScrollView tanıtılmıştır. ListView'ın en çok kullanılan yapı olduğu ancak son 4-5 senede yerini RecyclerView'a bıraktığı, bunun pek çok nedeni olduğu belirtilmiştir. RecyclerView'ın daha fazla özellik sunması ve performans artırma noktasında ciddi katkılar sağlaması temel nedenlerdir.

ListView ve RecyclerView arasındaki temel fark bir öğrenci tarafından doğru bir şekilde açıklanmıştır: ListView'da bin kişilik bir listeden bahsedildiğinde, tüm item'lar aktivite açıldığı anda yüklenir; RecyclerView'da ise aşağıya kaydırıldıkça item'lar yüklenir. Hoca bunu onaylamış, ekranda gösterilen on item'ın yüklendiğini, bir sonraki on item'ın henüz çekilmemiş olduğunu belirtmiştir. RecyclerView adından da anlaşılacağı üzere item'ları aşağı doğru ilerledikçe bazı item'lar ekranın görüntüsünden çıktığında, bu item view'lar sonraki view'lara geçildiğinde yeni gelen item'lar tarafından tekrar kullanılabilir. Bu recycling (geri dönüşüm) mantığı kaynak ve performans farkı yaratır; hem user experience (verinin görsel olarak sunulması) hem de kaynak tüketimi (RAM, pil) açısından büyük avantaj sağlar.

ViewHolder pattern açıklanmıştır. Her iki liste yapısının da kullanabildiği bu pattern, RecyclerView'da zorunlu, ListView'da opsiyoneldir. ListView'da ViewHolder pattern'i kullanılırsa performans önemli ölçüde RecyclerView'a yaklaşır, ancak RecyclerView hâlâ daha iyi performans gösterir. ListView'da ViewHolder kullanılmazsa ciddi kullanıcı memnuniyetsizliği ve kaynak tüketimi sorunu ortaya çıkar. Hoca, ViewHolder opsiyonelliğinin ListView'un en büyük sıkıntısı olduğunu vurgulamıştır.

LayoutManager kavramı detaylı olarak ele alınmıştır. ListView'da sadece dikeyde bir view oluşturma şansı varken, RecyclerView'da üç farklı layout kullanma opsiyonu vardır: Linear Layout Manager (hem yatay hem dikey), Grid Layout Manager (galeri uygulamaları için ideal, resimler arasında hızlı ilerleme), Staggered Layout Manager (Pinterest benzeri, farklı boyutlarda item düzeni). Windows Phone'un tile management yapısına yakın bir yapı sunduğu, farklı

görünümlemlerle kullanma şansı verdiği belirtilmiştir.

Item dekorasyonu ve animasyonu açısından ListView'in oldukça zayıf olduğu, opsiyonel olarak sunmamasının ötesinde customize etme imkanı da vermediği, RecyclerView'da ise item animator ve item decoration ile farklı item'lar tasarlanıp anime edilebileceği belirtilmiştir. Bu konunun karmaşıklığı da vurgulanmış, animation ve decoration işine girildiğinde belli oranda efor harcanması gerektiği söylenmiştir.

OnItemClickListener özelliği RecyclerView'a özel olarak tanıtılmıştır. Tek tip click işlemi yerine touch işleminde birden fazla touch tipini destekleyen listener ile item üzerinde farklı aksiyonlar tanımlanabilir. setHasFixedSize metodu ise item sayısı belli ve değişmezse kullanılır; örneğin dünyadaki 200 ülke ve her ülkedeki 100 şehir (toplam 20.000 item) gibi. Bu sayede RecyclerView her seferinde aynı pozisyonu alacağı için daha iyi kaynak planlaması yapılır ve cache mekanizması oluşturulur.

Hoca, "her an RecyclerView kullanılmalı mı" sorusuna da cevap vermiştir: Hayır. Çok basit durumlar (İstanbul, Ankara, İzmir, tüm Türkiye'nin şehirleri gibi), animasyon ihtiyacı olmayan, kompleks bir yapı oluşturulmayacak durumlarda basit bir ListView yeterlidir. Önemli olan ListView'da ViewHolder pattern'inin kullanılmasıdır.

RecyclerView ile liste yaratma adımları kod üzerinden açıklanmıştır. XML'de RecyclerView tanımlanır ve dependencies'e com.android.support:recyclerview-version-7 versiyonu eklenmelidir. Java tarafında RecyclerView bileşeni, Adapter ve LayoutManager tanımlanır. setContentView ile bağlama, findViewById ile ilişkilendirme, setHasFixedSize, setLayoutManager ve setAdapter çağrılarını yapar. Adapter kavramı detaylı olarak açıklanmıştır: view (ön yüz) ile data source arasındaki veri akışını sağlayan parça. Hoca, adapter ile controller kavramlarını ayırmıştır: Controller tüm işlemleri (veri çekme, işleme, ön yüze aktarma) yönetir, Adapter ise sadece RecyclerView/ListView'e özel verinin view'a aktarımını sağlar. Bir öğrencinin "controller adapter mı oluyor" sorusuna hoca, MVC'deki controller'ın tüm işlemleri yöneten kod, adapter'ın ise sadece veri aktarımını sağlayan yapı olduğunu açıkça belirtmiştir.

Adapter'ın temel metotları açıklanmıştır. onCreateViewHolder her item için bir ViewHolder üretir; LayoutInflater.from.inflate ile yeni bir item görüntüsü oluşturulur, burada item'ın dış kıyafeti XML dosyasıyla temsil edilir. onBindViewHolder oluşturulan view'un içine data source'dan gelen veriyi position'a göre set eder. getItemCount ise data set uzunluğunu döndürür. Bir öğrencinin "onCreateViewHolder içinde farklı tiplerde yapılar oluşturmak istersem nasıl tutarım" sorusuna hoca, eğer gerçekten farklı tipler gerekirse iki ayrı RecyclerView kullanılabileceğini, çünkü aynı RecyclerView içinde tüm item'ların aynı yapıda olması gerektiğini söylemiştir.

ViewHolder'ın dinamik oluşumu konusunda bir öğrencinin sorusu üzerine, "Creates only as many ViewHolders as are needed to display on screen portion of the dynamic content" ifadesi açıklanmıştır. RecyclerView sadece ekranda görünen kadar ViewHolder oluşturur, scroll edildikçe ekranın dışına çıkan item'ların ViewHolder'ları yeni gelen item'lar tarafından yeniden kullanılır. Bu temel prensibin dinamik bir şekilde gerçekleştiği vurgulanmıştır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.